Laboratorio di Calcolo per Fisici

Esercitazione n. 3, 26-27 Ottobre

Canale A-Dec 2026-2027, Docenti: G. Artoni, L. Caprini

Lo scopo della terza esercitazione di laboratorio è di scrivere da zero un programma in C usando le funzioni della libreria matematica `math.h` e di familiarizzare con i diversi comandi di formattazione dell'output per realizzare diversi tipi di grafico.

Fate il login utilizzando `lcgaXXX` sia come *userid* sia come *password*, dove `XXX` è il codice a tre cifre associato al vostro gruppo, così come indicato nella comunicazione dei gruppi.
In quanto segue, si assume che la vostra **`home`** sia `/home/studenti/lcgaXXX`. L'indirizzo potrebbe essere diverso, ma potete controllarlo immediatamente aprendo il terminale ed utilizzando il comando `pwd`.

---

1. Aprite una finestra di *terminale*.
2. Nella vostra home create una cartella chiamata `EX3` e posizionatevi all'interno di quest'ultima. Tutti i file, gli script e i codici C dovranno essere contenuti all'interno di questa cartella.

▶ Prima parte (obbligatoria)

Un moto circolare uniforme sul piano (x, y) è descritto dalle equazioni:

$$\begin{cases} x(t) = R\cos(\omega t) \\ y(t) = R\sin(\omega t) \end{cases},$$

dove R è il raggio della traiettoria e ω la velocità angolare. L'obiettivo della prima parte dell'esercitazione è quello di scrivere un codice in C contenuto nel file `circle.c` che utilizzi la libreria matematica del C per risolvere le equazioni della traiettoria per $R = 6.2$ m, $\omega = 0.1$ rad/s e stampi i risultati in maniera opportuna.

1. Scrivete un programma `circle.c` che chieda all'utente di inserire il valore del tempo t da tastiera, e usi il valore inserito per calcolare e stampare i valori di $x(t)$ e $y(t)$ in modo che sullo schermo compaia:

   ```
   (x,y)= ...,...
   ```

   (dove al posto dei puntini di sospensione bisogna stampare i valori ottenuti). Stampate i risultati con almeno 6 cifre di cui 4 dopo la virgola. Verificate che i risultati ottenuti con il programma riproducano quelli che si otterrebbero risolvendo a mano le equazioni del moto, ad esempio per i valori: $t = 0$ s, $t = 5$ s, $t = 10$ s.

Attenzione! Per sfruttare le funzioni della libreria matematica (come ad es. le funzioni seno e coseno) bisogna inserire prima del `main()` la riga di codice

```
#include <math.h>
```

e, in fase di compilazione, chiamare il compilatore `gcc` con l'opzione `-lm`, cioè:

```
$ gcc -Wall -o circle circle.c -lm
```

Un elenco (parziale) delle funzioni matematiche contenute in `math.h` si trova in fondo al testo dell'esercitazione.

Leggere un valore da tastiera: `scanf`. Finora i valori delle variabili erano scritti direttamente nel codice. Per farli inserire all'utente mentre il programma è in esecuzione si usa la funzione `scanf` (anch'essa in `stdio.h`), che è in un certo senso l'inverso di `printf`: legge dalla tastiera un valore nel formato indicato e lo memorizza in una variabile.

```
double t;
printf("Inserire il tempo t (in s): ");
scanf("%lf", &t);
```

Il programma si ferma sulla `scanf` finché l'utente non scrive un numero e preme Invio. Due dettagli importanti:

- per leggere un `double` il descrittore è `%lf` (in `scanf`, a differenza di `printf`, `%f` vale solo per i `float`); per un `int` si usa `%d`;
- davanti al nome della variabile va messo il simbolo `&`: `scanf` deve sapere *dove* si trova la variabile in memoria per poterci scrivere. Il significato preciso di `&` lo vedremo più avanti nel corso: per ora ricordate solo che in `scanf` ci vuole.

Dimenticare la `&` o usare `%f` con un `double` sono gli errori più comuni: compilate sempre con `-Wall`, perché il compilatore li segnala con un *warning*.

Suggerimento: Capiamo prima di tutto quali sono le variabili che dobbiamo definire per poter scrivere il codice. Le equazioni del moto circolare uniforme contengono R, ω e t: la prima equazione è associata alla coordinata x e la seconda equazione alla coordinata y, quindi avremo sicuramente bisogno di una variabile `x` e di una variabile `y` (di che tipo?) che contengano il risultato, rispettivamente, di $R\cos(\omega t)$ e $R\sin(\omega t)$. Abbiamo poi bisogno di una variabile in cui memorizzare il tempo introdotto dall'utente (chiamiamola `t`). Infine, per R e ω definiamo nel `main()` due variabili `R` e `omega`, inizializzate con i valori dati dal testo. Il codice avrà quindi la forma

```c
#include .... // file di intestazione per leggere e stampare dati e per
    le funzioni matematiche

int main() {
  // definizioni delle variabili
  double R = 6.2;    // raggio (m)
  double omega = ...; // velocita' angolare (rad/s)

  // richiesta di inserimento di dati
  printf(...); //
  // lettura dei dati
  scanf(...); //

  // calcolo delle equazioni del moto
  x = ...;
  y = ...;

  // stampa del risultato
  printf("(x,y): ...", ...);

  return 0;
}

```

Listato 1: Schema di `circle.c`

2. Una volta verificato che il programma produce risultati corretti, riformattate l'output in modo da stampare i tre valori `t`, `x`, `y` su tre colonne, separate da spazi vuoti, come nell'esempio sottostante (stampare ciascun valore con almeno 8 cifre di cui 4 dopo la virgola):

```
 2.5000   4.3841   4.3841

```

Suggerimento per tutti: Per modificare piccole parti di programma senza cancellare quello che si è fatto in precedenza può essere utile *commentare* le istruzioni che non ci sono utili.

► **Seconda parte (obbligatoria)**

In questa parte dell'esercitazione useremo il programma `circle.c` per studiare l'andamento del moto circolare uniforme, utilizzando python.

1. Eseguite il programma `circle.c` un numero di volte sufficiente ad avere una distribuzione di punti su tutta la traiettoria circolare. Salvate i risultati su un file `traiettoria.dat` che contenga tre colonne `t`, `x`, `y`.

Suggerimento: In questa seconda parte dobbiamo lanciare il programma più volte, introducendo ogni volta un valore diverso del tempo, e salvare i risultati su di un file. Invece di scrivere a mano dentro il file `traiettoria.dat` i valori di `t`, `x`, `y` che ad ogni esecuzione vengono stampati su schermo, possiamo utilizzare una strategia più conveniente: si può "redirigere l'output" di un programma su un file. Attenzione: nel codice abbiamo due occasioni in cui stampiamo qualcosa su schermo: la prima volta è per chiedere in input il valore del tempo al quale calcolare le equazioni del moto, la seconda volta invece è per stampare i valori del tempo, della `x` e della `y`. Se redirigiamo l'output su file, anche la domanda finirebbe nel file (e non la vedremmo più sullo schermo).
Per risolvere il problema bisogna sapere che ogni programma ha a disposizione *due* canali di output distinti, entrambi collegati di default al terminale:

- lo *standard output* (`stdout`), dove scrive `printf`, pensato per i *risultati* del programma;
- lo *standard error* (`stderr`), pensato per i *messaggi* rivolti all'utente (domande, avvisi, errori).

Sullo schermo i due canali sono indistinguibili, ma la redirezione con `>` (o `>>`) sposta su file *solo* lo standard output: ciò che è scritto su standard error continua ad apparire sul terminale. Useremo quindi `stderr` per la domanda e `stdout` per i valori di `t`, `x`, `y`.
Per stampare su "canali" diversi possiamo usare la funzione `fprintf`, che si comporta in maniera del tutto uguale a `printf`, con la sola (importante) differenza che il primo argomento deve indicare l'output su cui vogliamo stampare. Possiamo quindi usare la funzione `fprintf(stdout, "...", ...)` per stampare su standard output e `fprintf(stderr, "...", ...)` per stampare su standard error (`fprintf(stdout, ...)` è quindi del tutto equivalente a `printf(...)`).
Ecco quindi un possibile schema del codice:

```c
#include ... // files intestazione per leggere e stampare dati e per le
    funzioni matematiche

int main() {
  // definizioni delle variabili
  double R = 6.2;    // raggio (m)
  double omega = ...; // velocita' angolare (rad/s)

  // richiesta di inserimento di dati
  // usiamo stderr per separare questo output da quello del secondo
    printf
  fprintf(stderr, ...);
  // lettura dei dati
  scanf(...);

  // calcolo delle equazioni del moto
  x = ...;
  y = ...;
```

```c

  // stampa del risultato
  fprintf(stdout, "...", t, x, y);

  return 0;
}

```

Listato 2: Schema di `circle.c` con `stderr` e `stdout`

Quindi per redirigere i valori di `t`, `x`, `y` sul file `traiettoria.dat` basta usare il comando su terminale

```
$ ./circle >> traiettoria.dat
```

Attenzione! Il comando `>>` aggiunge ("appende") i dati al file dove vengono rediretti; ciò significa che i dati contenuti nel file non vengono sovrascritti, e i nuovi dati vengono inseriti in fondo al file. Se si vogliono sovrascrivere i dati con quelli nuovi usare:

```
$ ./circle > traiettoria.dat
```

2. Utilizzate python per graficare la traiettoria, graficando y in funzione di x. Migliorate l'aspetto del grafico aggiungendo i nomi sui due assi principali, un titolo e una legenda. Se necessario, modificate l'intervallo di visualizzazione dell'asse x e y.

   Suggerimento: Per graficare solo due colonne di un file che ne contiene molte, la funzione `loadtxt` va chiamata con il parametro `usecols=(i,j)`, dove `i` e `j` sono le colonne da graficare (partendo da 0, che indica la prima colonna). Nel nostro caso, se volessimo graficare solo la seconda e la terza colonna il comando completo da usare nello script python diventerebbe:

   ```
   np.loadtxt('traiettoria.dat', unpack=True, usecols=(1,2))
   ```

3. Create altri due grafici che mostrino l'andamento della coordinata x e della coordinata y in funzione del tempo, rispettivamente.

4. Salvate i tre grafici in tre file separati, chiamati: `traiettoria.png`, `x.png`, `y.png`. Per salvare i grafici con matplotlib, bisogna usare il seguente comando python

   ```
   plt.savefig("traiettoria.png")
   ```

Suggerimento per tutti: Per fare più grafici (in finestre separate) usando lo stesso script possiamo usare il comando `plt.figure(x)` dove `x` è un n. progressivo (ad es. 1 per la prima figura, 2 per la seconda figura, 3 per la terza figura ecc.). Lo script avrà dunque la seguente forma

```
import ... # importo il modulo matplotlib.pyplot con alias plt
import ... # importo il modulo numpy con alias np

# leggo dal file traiettoria.dat i valori del tempo, delle x e delle y
# e li associo a 3 vettori (t,x,y)
t, x, y = np.loadtxt(.....)

# creo la prima figura
plt.figure(1)
# titolo figura
# nomi degli assi
# grafico della figura
plt.plot(x, y, ...)
# creo la legenda sul grafico
# salvo il grafico in formato png

# creo la seconda figura
plt.figure(2)
# titolo figura
# nomi degli assi
# grafico della figura
plt.plot(t, x, ...)
# creo la legenda sul grafico
# salvo il grafico in formato png

# creo la terza figura
plt.figure(3)
# titolo figura
# nomi degli assi
# grafico della figura
plt.plot(t, y, ...)
# creo la legenda sul grafico
# salvo il grafico in formato png

# SOLO ALLA FINE, UNA VOLTA SOLA
plt.show()

```

Listato 3: Schema dello script python

Dato che la traiettoria deve essere circolare, ma python crea figure rettangolari, per impostare lo stesso rapporto tra asse x e asse y nel primo grafico, prima di `plt.plot(x, y, ...)` si può usare l'istruzione

```
plt.gca().set_aspect("equal")
```

► **Terza parte (facoltativa)**

L'andamento delle componenti della velocità in un moto circolare uniforme è dato da:

$$\begin{cases} v_x(t) = -\omega R \sin(\omega t) \\ v_y(t) = \omega R \cos(\omega t) \end{cases},$$

1. Fate una copia del programma `circle.c` e rinominatela `velocity.c`. Modificate quest'ultimo in modo che calcoli anche la velocità del punto lungo la traiettoria, e inserisca i risultati ottenuti nel file `traiettoria.dat`, aggiungendo altre due colonne oltre quelle esistenti.

2. Graficate l'andamento della velocità in funzione del tempo lungo la traiettoria: quanto vale il modulo della velocità? In che direzione punta il vettore velocità per $t = 0$ s? E per $t = 10$ s? In quali punti sono massimi/minimi i valori di $v_x(t)$ e $v_y(t)$? Qual è il significato fisico di quello che osservate? Queste domande servono a farvi riflettere su quello che state facendo: non dovete scrivere le risposte da nessuna parte!

3. Disegnate il vettore velocità lungo la traiettoria come una freccia che punta nella direzione corretta sfruttando il seguente codice python che disegna `len(x)` frecce di lunghezza `(vx,vy)` nei punti `(x,y)`:

```
x, y, vx, vy = np.loadtxt('traiettoria.dat', usecols=(1,2,3,4), unpack=True)
for cc in range(0, len(x)):
    xi = x[cc]
    yi = y[cc]
    vxi = vx[cc]
    vyi = vy[cc]
    plt.arrow(xi, yi, vxi, vyi, width=0.2, head_width=0.5, head_length=0.3,
   fc='r', ec='r')

```

▶ Funzioni più comuni della libreria `math.h`:

Se non indicato diversamente, le funzioni prendono un solo parametro.

`acos`	arcocoseno	`frexp`	frazione e potenza di due
`asin`	arcoseno	`ldexp`	operazione in virgola mobile
`atan`	arcotangente	`log`	logaritmo naturale
`atan2(x, y)`	arcotangente di due parametri	`log10`	logaritmo in base 10
`ceil`	il più piccolo intero non minore del parametro	`pow(x, y)`	eleva un valore dato ad esponente, x^y
`cos`	coseno	`sin`	seno
`cosh`	coseno iperbolico	`sinh`	seno iperbolico
`exp`	funzione esponenziale, calcola e^x	`sqrt`	radice quadrata
`fabs`	valore assoluto	`tan`	tangente
`floor`	il più grande intero non maggiore del parametro	`tanh`	tangente iperbolica
`fmod`	resto del numero in virgola mobile		